

Kubernetes Project

Documented by: NISHAT AFLA

What I did in this project:

- Deployed Dockerized Flask applications on Kubernetes using Minikube.
- Designed and customized Helm charts for reusable Kubernetes deployments.
- Used values.yaml to manage multiple applications dynamically within a single Helm chart.
- Configured custom hostnames and Minikube networking.
- Gained hands-on experience in Kubernetes orchestration, Helm templating, and scalable application deployment.

Deploy multiple Flask applications on Kubernetes using Helm chart-

first we used NodePort service. previous task:

https://dev.azure.com/fronturetech/Fronture%20ERP/_wiki/wikis/Fronture-ERP.wiki/349/Helm

****Nodeport Limitations****

- Each app requires a unique port.
- Not scalable for many apps.
- No path-based routing.

now we are migrating to Ingress for better routing and scalability.

Ingress Service with Multiple Apps****

Now we will-

- Deploy two Flask apps using a single Helm chart.
- Use Ingress for routing based on URL paths.
- Manage everything using variables in values.yaml.

****Structure****

****values.yaml:**** https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-helloApp-chart/values.yaml

apps:

```
- name: hello1
image: flask-hello1:h1
containerPort: 5000
path: /hello1
```

```
- name: hello2
image: flask-hello2:h2
containerPort: 5000
path: /hello2
```

****Key Templates****

****deployment.yaml:**** https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-helloApp-chart/templates/deployment.yaml

```
{- range .Values.apps } } //loop through multiple app
apiVersion: apps/v1
kind: Deployment
metadata:
name: { { .name } }
spec:
replicas: 1
selector:
matchLabels:
app: { { .name } }
template:
metadata:
labels:
app: { { .name } }
spec:
containers:
- name: { { .name } }
image: { { .image } }
ports:
- containerPort: { { .containerPort } }
```

```
{ { - end } }
```

****service.yaml:**** https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-helloApp-chart/templates/service.yaml

```
{- range .Values.apps } } //loop through multiple app
apiVersion: v1
kind: Service
metadata:
name: { { .name } }
spec:
selector:
app: { { .name } }
```

```
ports:
- protocol: TCP
port: 80
targetPort: {{ .containerPort }}
```

```
{{- end }}
```

****ingress.yaml:**** https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-helloApp-chart/templates/ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: flask-ingress
spec:
rules:
- http:
paths:
{{- range .Values.apps }}
- path: {{ .path }}
pathType: Prefix
backend:
service:
name: {{ .name }}
port:
number: 80
{{- end }}
```

****_Now Apps Access Method_****

****Enable ingress:****

>minikube addons enable ingress

Our customized host name: hello-app.local

minikube Ip : 192.168.59.100

for setting customized host name :

Add the hosts entry:

echo "192.168.59.100 hello-app.local" | sudo tee -a /etc/hosts

```
nishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/Flask$ cat /etc/hosts
127.0.0.1 localhost
127.0.1.1 nishat-Lenovo-ideapad-320-14IKB

# The following lines are desirable for IPv6 capable hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters

192.168.59.100  hello-app.local
nishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/Flask$
```

****Remember to add this host name at value.yaml file****

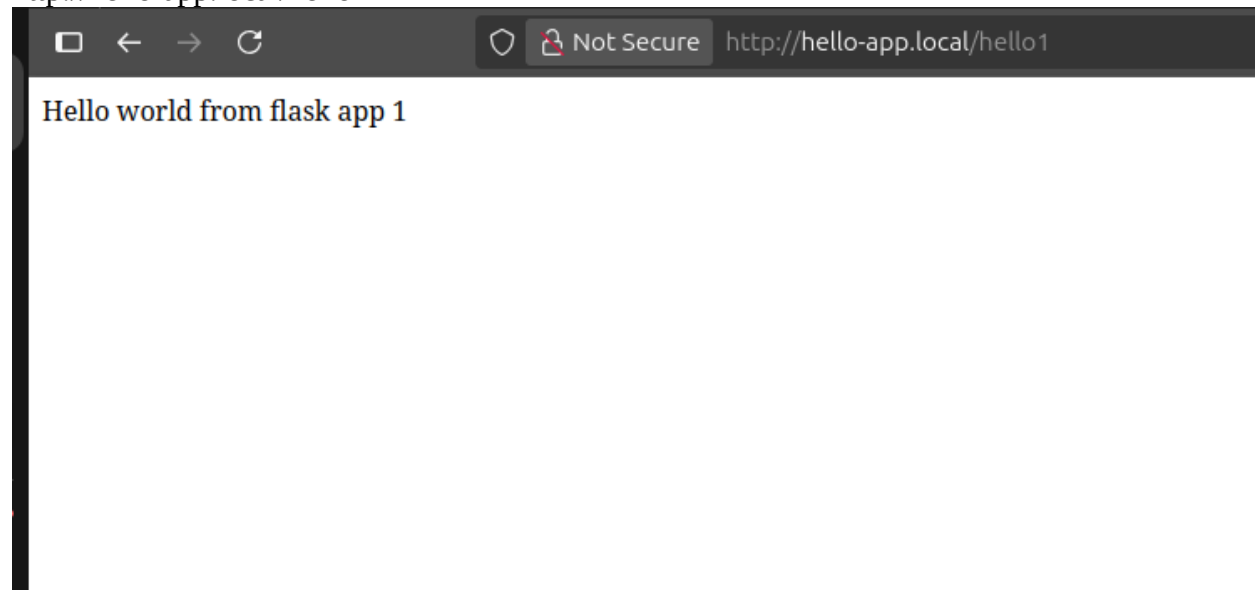
>ingress:

host: "hello-app-local.dev"

****Then access via:****

http://hello-app.local/hello1

http://hello-app.local/hello2



****Advantages****

- One Ingress handles multiple apps.
- Clean path-based routing.
- Easier to scale and manage.

Deployment for NGINX as a reverse proxy

Prerequisites:

Before we begin, we should ensure that we have the following:

- * A machine running Linux, macOS, or Windows.
- * A hypervisor, such as VirtualBox or KVM, is installed on our machine. If we're using Windows, we can use Hyper-V instead.
- * A shell environment, such as Bash or PowerShell.
- * curl installed on our machine.

Install Minikube:

For Linux run:

```
curl -Lo minikube https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

```
chmod +x minikube
```

```
sudo mv minikube /usr/local/bin/
```

Install VirtualBox:

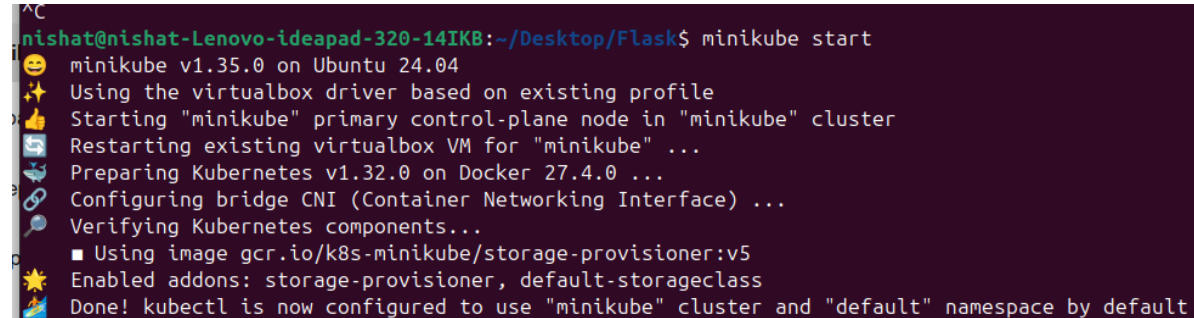
for linux

```
sudo apt install virtualbox virtualbox-ext-pack
```

Install `kubectl`:

```
sudo apt install -y kubectl
```

start minikube



```
nishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/Flask$ minikube start
🐳 minikube v1.35.0 on Ubuntu 24.04
🌟 Using the virtualbox driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
🔄 Restarting existing virtualbox VM for "minikube" ...
🔧 Preparing Kubernetes v1.32.0 on Docker 27.4.0 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌞 Enabled addons: storage-provisioner, default-storageclass
🏡 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

1.Create a Simple Flask App

`app.py`:

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return "Hello, Flask!"
```

```
def hello():  
    return "Hello from Flask in Kubernetes!"
```

```
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

```
**`requirements.txt`:**
```

```
Flask==2.2.2
```

```
* * *
```

2.Dockerize the Flask App

```
**`Dockerfile`:**
```

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY app.py .
```

```
CMD ["python", "app.py"]
```

Build Docker Image in Minikube

Set your environment to Minikube:

```
eval $(minikube docker-env)
```

Then build the image:

```
docker build -t flask-app:latest .
```

Check that it's available:

```
docker images
```

* * *

3.Kubernetes Manifests

A. Flask Deployment & Service

`flask-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask
  template:
    metadata:
      labels:
        app: flask
    spec:
      containers:
        - name: flask
          image: flask-app:latest
          ports:
            - containerPort: 5000
```

```
apiVersion: v1
kind: Service
metadata:
  name: flask-service
spec:
  selector:
    app: flask
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
```

Apply:

```
kubectl apply -f flask-deployment.yaml
```

* * *

B. NGINX ConfigMap

```
**`nginx-config.yaml`:**
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  default.conf: |
    server {
      listen 80;
      location / {
        proxy_pass http://flask-service:5000;
      }
    }
  }
```

Apply:

```
kubectl apply -f nginx-config.yaml
```

* * *

C. NGINX Deployment & Service

```
**`nginx-deployment.yaml`:**
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
```



```

    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
    volumeMounts:
    - name: nginx-config-volume
      mountPath: /etc/nginx/conf.d
  volumes:
  - name: nginx-config-volume
    configMap:
      name: nginx-config
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080

```

Apply:

```
kubectl apply -f nginx-deployment.yaml
```

* * *

4. Access the App

Get the Minikube IP:

```
minikube ip
```

Visit the app in browser:

http://<minikube-ip>:30080

You should see the result:



The screenshot shows a web browser window with the address bar displaying 'http://192.168.59.100:30080'. The page content says 'Hello from Flask in Kubernetes!'. Below the browser window, a terminal window shows the command 'kubectl get pods' and its output:

NAME	READY	STATUS	RESTARTS	AGE
flask-deployment-5b7756f58b-p8dxh	1/1	Running	1 (2m9s ago)	2d22h
nginx-deployment-7f4b584784-pbdwl	1/1	Running	1 (2m9s ago)	3d16h

The terminal prompt is 'nishat@nishat-Lenovo-ideapad-320-14IK8:~/Desktop/Flask\$'. Below the terminal output, the text '**Git Url:**' is followed by the URL 'https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?version=GBKubernetes&_a=contents'.

****references**:**

<https://www.baeldung.com/ops/minikube-getting-started>

<https://medium.com/@kadampritesh46/getting-started-with-kubernetes-a-step-by-step-guide-using-minikube-e88867296bcb>

Deploy flask app with helm using NodePort service

**Prerequisites**

Make sure the following are installed and configured:

- [] Docker

- [] Minikube

- [] Helm

- [] kubectl

****A Flask app already created and Dockerized**** ****and Minikube is started in previous part****

https://dev.azure.com/fronturetech/Fronture%20ERP/_wiki/wikis/Fronture-ERP.wiki/347/Deploying-flask-app-using-kubernetes

Note: If you build docker image directly, the image will be registered to your computer local docker repository, which cannot be accessed by the Minikube. Therefore, we have to build the image into Minikube's docker repository.

****Now A Helm chart was created using:****

```
>helm create flask-app-helm
```

This creates a directory structure like:

```
flask-app-helm/
├── charts/
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── hpa.yaml (optional)
│   ├── ingress.yaml (optional)
│   └── serviceaccount.yaml
├── values.yaml
└── Chart.yaml
```

****Customize Helm Chart****
_ ****in Chart.yaml file**** _

Edit metadata fields like name and version.

```
>apiVersion: v2
name: flask-app-helm
description: A Helm chart for Flask application
version: 0.1.0
```

https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-app-helm/Chart.yaml

_ ****in values.yaml**** _

Edit content of values.yaml:

```
>replicaCount: 1
image:
repository: flask-app
tag: latest
pullPolicy: Never
service:
type: NodePort
port: 5000
```

nodePort: 30088

https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-app-helm/values.yaml

**in deployment.yaml**

Ensure image and port match your Flask app Docker config.

https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-app-helm/templates/deployment.yaml

**Install Helm Chart**

Navigate to the directory containing the Helm chart:

```
cd ~/Desktop/Flask/flask-app-helm
```

Now Install the Helm chart:

```
>>helm install flask-app . --create-namespace --namespace flask-app
```

If successful, you'll see output including deployment notes.

```
nishat@nishat-Lenovo-Ideapad-320-14IKB: ~/Desktop/Flask/flask-app-helm$ helm install flask-app . --create-namespace --namespace flask-app
NAME: flask-app
LAST DEPLOYED: Tue May 20 11:53:02 2025
NAMESPACE: flask-app
STATUS: deployed
REVISION: 1
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace flask-app -o jsonpath="{.spec.ports[0].nodePort}" services flask-app-flask-app-helm)
  export NODE_IP=$(kubectl get nodes --namespace flask-app -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
nishat@nishat-Lenovo-Ideapad-320-14IKB: ~/Desktop/Flask/flask-app-helm$ export NODE_PORT=$(kubectl get --namespace flask-app -o jsonpath="{.spec.ports[0].nodePort}" services flask-app-flask-app-helm)
nishat@nishat-Lenovo-Ideapad-320-14IKB: ~/Desktop/Flask/flask-app-helm$ export NODE_IP=$(kubectl get nodes --namespace flask-app -o jsonpath="{.items[0].status.addresses[0].address}")
nishat@nishat-Lenovo-Ideapad-320-14IKB: ~/Desktop/Flask/flask-app-helm$ echo http://$NODE_IP:$NODE_PORT
http://192.168.59.100:32178
nishat@nishat-Lenovo-Ideapad-320-14IKB: ~/Desktop/Flask/flask-app-helm$ cd
```

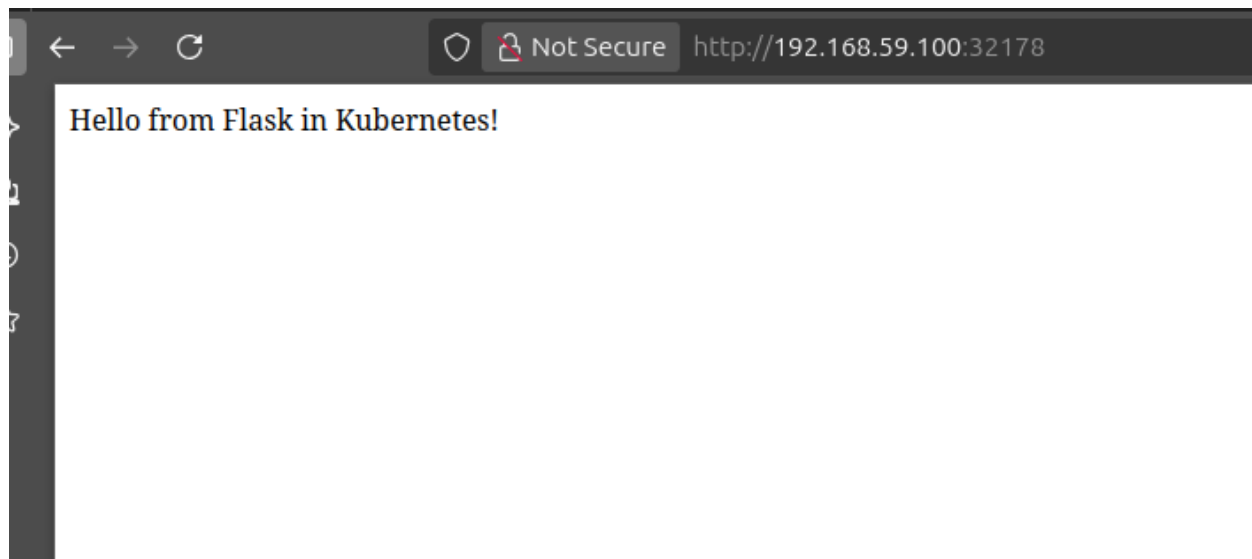
Run the following to get the service IP and port:

```
>export NODE_PORT=$(kubectl get --namespace flask-app -o jsonpath="{.spec.ports[0].nodePort}" services flask-app-flask-app-helm)
```

```
>export NODE_IP=$(kubectl get nodes --namespace flask-app -o jsonpath="{.items[0].status.addresses[0].address}")
```

```
>echo http://NODE_IP:$NODE_PORT
```

Open the output URL in your browser. You should see the Flask app running.



Additional Test:

**Helm Testing**

Helm lets you write simple Kubernetes Jobs to verify your app after installation. This is useful for CI/CD and production readiness.

>helm test flask-app --namespace flask-app

```
nishat@nishat-Lenovo-Ideapad-320-14IK8:~/Desktop/Flask/flask-app-helm$ helm test flask-app --namespace flask-app
NAME: flask-app
LAST DEPLOYED: Tue May 20 11:53:02 2025
NAMESPACE: flask-app
STATUS: deployed
REVISION: 1
TEST SUITE: flask-app-flask-app-helm-test-connection
Last Started: Thu May 22 13:07:50 2025
Last Completed: Thu May 22 13:08:06 2025
Phase: Succeeded
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace flask-app -o jsonpath="{.spec.ports[0].nodePort}" services flask-app-flask-app-helm)
  export NODE_IP=$(kubectl get nodes --namespace flask-app -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
```

**Multiple Environments (dev, staging, prod)**

Helm enables us to just use one template for different environments like dev, test, prod etc. based to different _values.yaml_ for platform resources

Create files:

* `values-dev.yaml` : https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-app-helm/values-dev.yaml

* `values-staging.yaml` : https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-app-helm/values-staging.yaml

* `values-prod.yaml` : https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-app-helm/values-prod.yaml

****Install helm for each environment:****

`helm install flask-dev ./flask-app-helm --namespace dev --create-namespace -f values-dev.yaml`

```
`helm install flask-staging ./flask-app-helm --namespace staging --create-namespace -f values-staging.yaml`
```

```
`helm install flask-prod ./flask-app-helm --namespace prod --create-namespace -f values-prod.yaml`
```

****Check helm releases:****

```
`helm list -A`
```

****Verify They Are Working****

```
>kubectl get all -n dev
```

```
kubectl get all -n staging
```

```
kubectl get all -n prod
```

You can access each app using Minikube and its assigned NodePort:

```
>minikube service --url -n dev flask-dev
```

```
minikube service --url -n staging flask-staging
```

```
minikube service --url -n prod flask-prod
```

you can upgrade your helm release. for example here we are updating staging release.

```
>helm upgrade flask-staging . -n staging -f values-staging.yaml
```

```
nishat@nishat-Lenovo-ideapad-320-14IK8:~/Desktop/Flask/Flask-app-helm$ helm upgrade flask-staging . -n staging -f values-staging.yaml
Release "flask-staging" has been upgraded. Happy Helming!
NAME: flask-staging
LAST DEPLOYED: Thu May 22 14:01:21 2025
NAMESPACE: staging
STATUS: deployed
REVISION: 2
NOTES:
```

**Summary**

Using Helm, you packaged your Flask application into a chart and deployed it to Minikube. This method abstracts the complexity of raw YAML files and provides better maintainability and environment flexibility through values.yaml.

Repo:https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/flask-app-helm

reference: <https://siweheee.medium.com/deploy-your-programs-onto-minikube-with-docker-and-helm-a68097e8d545>